

Scalable Web Application with Load Balancing

Project: Scalable Web Application with Load Balancing

Services: EC2, ELB, Auto Scaling, RDS, S3, Route 53

Goal: Deploy a web application that automatically scales based on traffic load with secure database connectivity and static file hosting.

Skills: VPC, Security Groups, EC2 Launch Templates, User Data Scripts, Load Balancing, Auto Scaling.

Definition:

The Scalable Web Application with Load Balancing project is designed to deploy a highly available, fault-tolerant, and scalable web application infrastructure on AWS. The architecture uses Elastic Load Balancer (ELB) to distribute incoming traffic across multiple EC2 instances running in an Auto Scaling Group. Amazon RDS provides managed database services, while Amazon S3 hosts static application content such as images, CSS, and JavaScript files. Route 53 is used for DNS management and routing user requests to the application.

Scope of the Project:

- Deploy scalable EC2 instances using Auto Scaling Groups.
- Configure Application Load Balancer to distribute traffic efficiently.
- Host application static files in Amazon S3.
- Deploy managed relational database using Amazon RDS.
- Configure Route 53 for domain management and traffic routing.
- Implement secure networking using VPC, subnets, route tables, and security groups.
- Enable high availability using Multi-AZ deployment.

Methodology:

1. AWS Native Scalable Architecture - Best Method

- Uses AWS managed services including EC2, ELB, Auto Scaling, RDS, S3, and Route 53.
- Provides automatic scaling based on CPU utilization and traffic load.
- Improves application availability and fault tolerance.
- Supports secure and isolated networking using VPC.
- Production-ready and suitable for enterprise deployment.
- Cost-effective and scalable according to workload requirements.

2. Single EC2 Traditional Hosting – Alternative Method

- Uses a single EC2 instance to host the application.
- Requires manual scaling and maintenance.
- No automatic failover or load balancing.
- Lower initial setup complexity but limited scalability.
- Higher risk of downtime during heavy traffic or server failure.
- Suitable for small testing environments only.

AWS Services used in the Project

1. Amazon EC2 (Elastic Compute Cloud)

- Definition: Amazon EC2 provides resizable virtual servers in the cloud.
- Purpose: Hosts the web application.
- Role: Runs application code and handles user requests.

2. Elastic Load Balancer (ELB)

- Definition: ELB automatically distributes incoming application traffic across multiple EC2 instances.
- Purpose: Improves application availability and scalability.
- Role: Balances traffic among EC2 instances.

3. Amazon EC2 Auto Scaling

- Definition: Auto Scaling automatically adjusts EC2 capacity based on demand.
- Purpose: Ensures application scalability and performance.
- Role: Launches or terminates EC2 instances automatically.

4. Amazon RDS (Relational Database Service)

- Definition: Amazon RDS is a managed relational database service.
- Purpose: Stores application data securely.
- Role: Provides scalable and highly available database services.

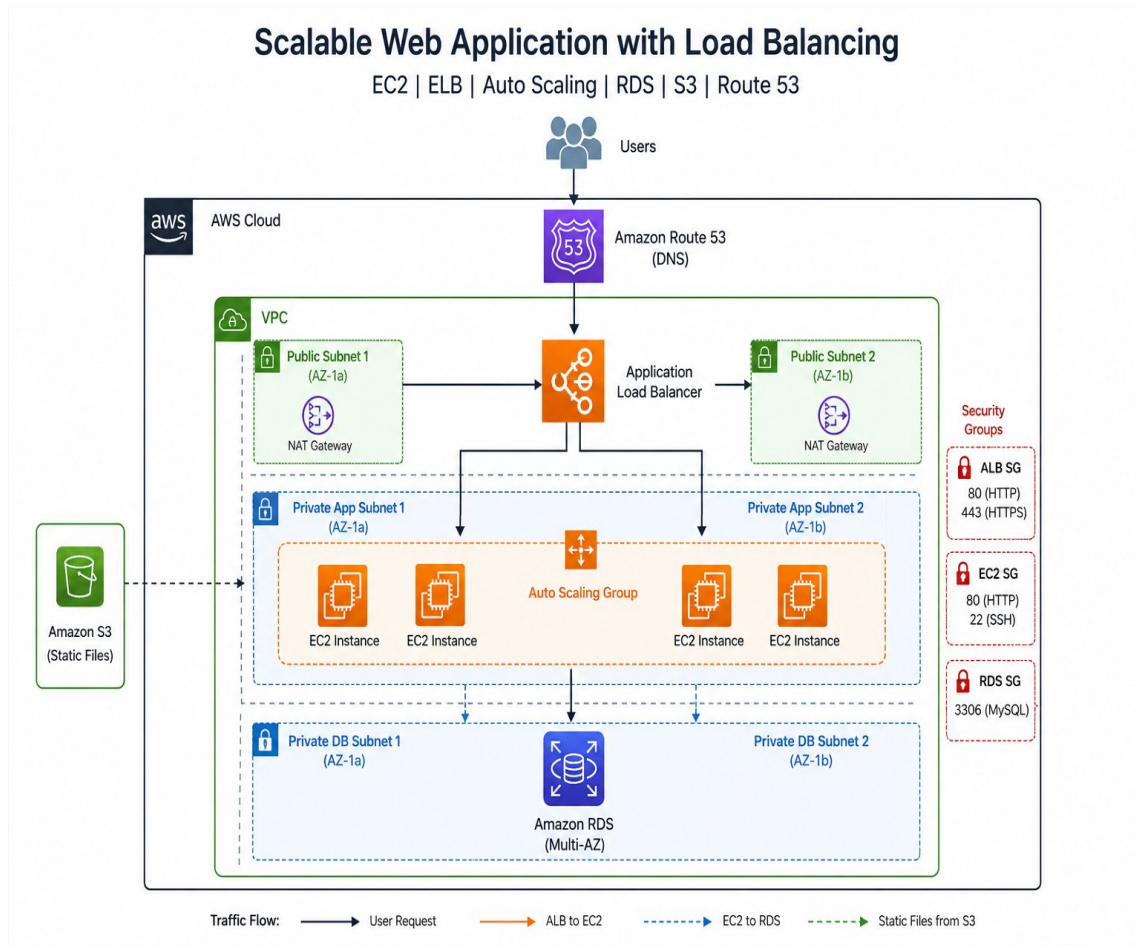
5. Amazon S3 (Simple Storage Service)

- Definition: Amazon S3 is an object storage service.
- Purpose: Stores static files such as images, CSS, and JavaScript.
- Role: Hosts static application content.

6. Amazon Route 53

- Definition: Route 53 is a scalable DNS and domain management service.
- Purpose: Routes users to the application.
- Role: Maps domain names to the load balancer.

Architecture Diagram:



User → Route 53 → Application Load Balancer → Auto Scaling Group → EC2 Instances → Amazon RDS
Static Files → Amazon S3

Implementation Steps to do the Project:

Step 1: Create VPC and Networking Components

- Go to AWS Console → VPC → Create VPC.
- Create Public and Private Subnets across multiple Availability Zones.
- Create Internet Gateway and attach it to the VPC.
- Configure Route Tables for public and private subnets.
- Enable internet access for public subnets.

← → ↻ ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#VpcDetails:VpcId=vpc-094e787c6727d5b85 ☆ ⓘ 🔍 M

aws [Search] [Alt+S] Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

VPC > Your VPCs > vpc-094e787c6727d5b85 ⓘ

VPC dashboard

AWS Global View ⓘ

Filter by VPC ▾

▼ Virtual private cloud

- Your VPCs
- Subnets
- Route tables
- Internet gateways
- Egress-only internet gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections

vpc-094e787c6727d5b85 / projectweb-vpc-vpc

Actions ▾

Details Info

VPC ID vpc-094e787c6727d5b85	State Available	Block Public Access Off	DNS hostnames Enabled
DNS resolution Enabled	Tenancy default	DHCP option set dopt-0354b00bba661b75e	Main route table rtb-071743cae6a8a8fc5
Main network ACL acl-07fdf481fcb7299f	Default VPC No	IPv4 CIDR 10.0.0.0/16	IPv6 pool -
IPv6 CIDR (Network border group) -	Network Address Usage metrics Disabled	Route 53 Resolver DNS Firewall rule groups -	Owner ID 973629900070
Encryption control ID -	Encryption control mode -		

Resource map

CIDRs

Flow logs

Tags

Integrations

CloudShell Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Step 2: Configure Security Groups

- Create Security Group for Load Balancer allowing HTTP and HTTPS traffic.
- Create Security Group for EC2 instances allowing traffic only from the Load Balancer.
- Create Security Group for RDS allowing database access from EC2 instances.

aws [Search] [Alt+S] Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

EC2 > Security Groups > sg-0a848766477b58f29 - rds-sg ⓘ

EC2

Dashboard

AWS Global View ⓘ

Events

▼ Instances

- Instances
- Instance Types
- Launch Templates
- Spot Requests
- Savings Plans
- Reserved Instances
- Dedicated Hosts
- Capacity Reservations
- Capacity Manager [New](#)

▼ Images

sg-0a848766477b58f29 - rds-sg

Actions ▾

Details

Security group name rds-sg	Security group ID sg-0a848766477b58f29	Description allow mysql	VPC ID vpc-094e787c6727d5b85 ⓘ
Owner 973629900070	Inbound rules count 1 Permission entry	Outbound rules count 1 Permission entry	

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (1)

Manage tags Edit inbound rules

Search

☐	Name	Security group rule ID	IP version	Type	Protocol
☐	-	sgr-03cdad21e3040b928	-	MYSQL/Aurora	TCP

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

sg-0e4cab2ee8d2f5f42 - ec2-sg

Actions

Details

Security group name

ec2-sg

Security group ID

sg-0e4cab2ee8d2f5f42

Description

allow port 80 only from alb-sg

VPC ID

vpc-094e787c6727d5b85

Owner

973629900070

Inbound rules count

1 Permission entry

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (1)

Manage tags

Edit inbound rules

Search

Name	Security group rule ID	IP version	Type	Protocol
-	snr-0202ear4re60da480	-	HTTP	TCP

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

sg-08b0a4eabba02b8de - alb-sg

Actions

Details

Security group name

alb-sg

Security group ID

sg-08b0a4eabba02b8de

Description

allow port 80 and 443

VPC ID

vpc-094e787c6727d5b85

Owner

973629900070

Inbound rules count

2 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)

Manage tags

Edit inbound rules

Search

Name	Security group rule ID	IP version	Type	Protocol
-	sgr-0c298ed7610334ded	IPv4	HTTP	TCP
-	sgr-0ec74160f65ac11f0	IPv4	HTTPS	TCP

Step 3: Create EC2 Launch Template

- Go to EC2 → Launch Templates → Create Launch Template.
- Choose Amazon Linux 2 AMI.
- Select instance type t2. micro.
- Attach EC2 Security Group.
- Add User Data script to install Apache web server.
- Step 4: Add User Data Script
- Install Apache using yum.
- Start and enable Apache service.
- Create sample HTML webpage.
- Save the Launch Template.

The screenshot displays the AWS Management Console interface for a Launch Template. The top navigation bar shows the AWS logo, a search bar, and the user's profile (Maheshwari). The left sidebar contains navigation links for EC2, Launch templates, and Elastic Block Store. The main content area is titled 'webapptemplate (lt-006fe58bfa8602499)' and includes a 'Launch template details' section with fields for Launch template ID, Launch template name, Default version, and Owner. Below this is a 'Launch template version details' section with a dropdown for Version (1 (Default)) and a table for Instance details (AMI ID, Instance type, Availability Zone, Availability Zone Id).

webapptemplate (lt-006fe58bfa8602499)

Launch template details

Launch template ID lt-006fe58bfa8602499	Launch template name webapptemplate	Default version 1	Owner arn:aws:iam::973629900070:root
--	--	----------------------	---

Launch template version details

Version: 1 (Default)

Date created: 2026-05-19T18:45:31.000Z

Created by: arn:aws:iam::973629900070:root

Instance details

AMI ID ami-09ed39e30153c3bf9	Instance type	Availability Zone	Availability Zone Id
---------------------------------	---------------	-------------------	----------------------

Step 5: Create Application Load Balancer

- Go to EC2 → Load Balancers → Create Load Balancer.
- Select Application Load Balancer.
- Choose internet-facing option.
- Select public subnets.
- Attach Load Balancer Security Group.

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Maheshwari (973629900070)

Maheshwari

EC2 > Load balancers > webloadbalancer

webloadbalancer

Details

▼ Details

Load balancer type

Application

Scheme

Internet-facing

Status

Active

Hosted zone

Zone ID: ZP97RAFLXTNZK

VPC

VPC ID: vpc-094e787c6727d5b85

Availability Zones

subnets: subnet-09bd7547dc63d1462, subnet-028b66e7357404d1c

Load balancer IP address type

IPv4

Date created

May 20, 2026, 00:01 (UTC+05:30)

Load balancer ARN

arn:aws:elasticloadbalancing:ap-south-1:973629900070:loadbalancer/app/webloadbalancer/925fb2c2c0a09095

DNS name

webloadbalancer-197237757.ap-south-1.elb.amazonaws.com

Step 6: Create Target Group

- Create Target Group for EC2 instances.
- Configure health check path as /.
- Attach Target Group to Load Balancer.

The screenshot displays the AWS Management Console interface for the 'Target groups' section. The left-hand navigation pane includes categories like 'Elastic Block Store', 'Network & Security', 'Load Balancing', and 'Auto Scaling'. The main area is titled 'Target groups (1/1)' and contains a table listing the target groups. A single target group, 'web-app-tg', is listed with its ARN, port (80), protocol (HTTP), and target type (Instance). Below the table, the 'Details' tab is active, showing the specific configuration for the 'web-app-tg' target group, including its full ARN.

Name	ARN	Port	Protocol	Target type
web-app-tg	arn:aws:elasticloadbalancing:ap-south-1:973629900070:targetgroup/web-app-tg/2fa5c26942aea733	80	HTTP	Instance

Step 7: Create Auto Scaling Group

- Go to EC2 → Auto Scaling Groups → Create Auto Scaling Group.
- Select Launch Template.
- Choose private subnets.
- Attach Application Load Balancer.
- Configure scaling policy based on CPU utilization.

The screenshot displays the AWS Management Console interface for an Auto Scaling Group named 'webappsg'. The left-hand navigation pane is expanded, showing categories like 'Network & Security', 'Load Balancing', and 'Auto Scaling'. The 'Auto Scaling' section is selected, and the 'webappsg' group is chosen. The main content area shows the 'webappsg Capacity overview' with an 'Edit' button. Below this, a table lists the group's details: 'Desired capacity' is 1, 'Scaling limits' are 1 - 5, 'Desired capacity type' is 'Units (weights)', and 'Status' is 'At desired capacity'. The 'Date created' is 'Wed May 20 2026 00:40:03 GMT+0530 (India Standard Time)'. A tabbed interface at the bottom shows 'Details' as the active tab, with other tabs including 'Integrations', 'Automatic scaling', 'Instance management', 'Instance refresh', and 'Activity'. Below the tabs, a 'Launch template' section is visible, with an 'Edit' button and a table header showing 'Launch template', 'AMI ID', 'Instance type', and 'Owner'.

Desired capacity	Scaling limits	Desired capacity type	Status
1	1 - 5	Units (weights)	At desired capacity

Launch template	AMI ID	Instance type	Owner
-----------------	--------	---------------	-------

Step 8: Create Amazon RDS Database

- Go to RDS → Create Database.
- Choose MySQL engine.
- Select Free Tier template.
- Configure database username and password.
- Attach RDS Security Group.

aws [Search] [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

Aurora and RDS > Databases > database-1

Aurora and RDS

- Dashboard
- Databases**
- Performance insights
- Snapshots
- Exports in Amazon S3
- Automated backups
- Reserved instances
- Proxies

Subnet groups
Parameter groups
Option groups
Custom engine versions
Zero-ETL integrations

Connectivity & security | Monitoring | Logs & events | Configuration | Zero-ETL integrations

Connect using | Info

- ☒ **Code snippets**
Use when connecting through SDK, APIs, or third-party tools including agents.
- ☐ **CloudShell**
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.
- ☐ **Endpoints**
Use when connecting through any IDE interface.

Internet access gateway
☒ Disabled

IAM Authentication
☒ Disabled

Programming language | **Endpoint type**

MySQL (macOS) | Instance endpoint

Connection steps
Follow the steps below to paste the code of each step in your tool and run the commands. The snippets dynamically reflect the authentication configuration.

```
1 curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

Step 9: Create Amazon S3 Bucket

- Go to S3 → Create Bucket.
- Enable static website hosting.
- Upload images, CSS, and JavaScript files.

aws [Search] [Alt+S] Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

Amazon S3 > Buckets

Amazon S3

- ▼ **Buckets**
 - General purpose buckets**
 - Directory buckets
 - Table buckets
 - Vector buckets
- ▼ **Files**
 - File systems [New](#)
- ▼ **Access management and security**
 - Access Points
 - Access Points for FSx
 - Access Grants
 - IAM Access Analyzer
- ▼ **Storage management and**

Buckets

General purpose buckets | All AWS Regions | Directory buckets

General purpose buckets (1) | Info | Copy ARN | Empty | Delete | **Create bucket**

Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	Creation date
☐ my-unique-webapp-static-assets	Asia Pacific (Mumbai) ap-south-1	May 19, 2026, 19:05:12 (UTC+05:30)

► **Account snapshot** | Info | View dashboard
Updated daily
Storage Lens provides visibility into storage usage and activity trends.

► **External access summary** | Info
Updated daily
External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

Step 10: Configure Route 53

- Go to Route 53 → Hosted Zones.
- Create Hosted Zone for the domain.
- Create A record pointing to Application Load Balancer.

The screenshot shows the AWS Route 53 console. The left sidebar contains navigation links: Dashboard, Hosted zones (selected), Health checks, Profiles, Global Resolver, VPC Resolver, and Domains. The main content area is titled 'Hosted zones (1)' and includes buttons for 'View details', 'Edit', 'Delete', and 'Create hosted zone'. Below these is a search bar and a table with one entry: 'sample.com' of type 'Public' created by 'Route 53'. The right sidebar shows '0 hosted zone selected' and a message to 'Select a hosted zone to see its details'.

Route 53

Dashboard
Hosted zones
Health checks
Profiles

▼ **Global Resolver**
Global resolvers [New](#)

▼ **VPC Resolver**
VPCs
Inbound endpoints
Outbound endpoints
Rules
Query logging
Outposts

▼ **Domains**

Hosted zones (1)

[View details](#) [Edit](#) [Delete](#) [Create hosted zone](#)

Automatic mode is the current search behavior optimized for best filter results. [To change modes go to settings.](#)

	Hosted zone name	Type	Created by
☐	sample.com	Public	Route 53

0 hosted zone selected

Select a hosted zone to see its details

The screenshot shows the AWS Route 53 console with the 'Records' tab selected for the 'sample.com' hosted zone. The left sidebar is the same as the previous screenshot. The main content area is titled 'Records (3)' and includes buttons for 'Delete record', 'Import zone file', and 'Create record'. Below these is a search bar and a table with one entry: 'sample.com' of type 'NS' with a 'Simple' routing policy. The right sidebar shows '0 records selected' and a message to 'Select a record to see its details'.

Route 53

Dashboard
Hosted zones
Health checks
Profiles

▼ **Global Resolver**
Global resolvers [New](#)

▼ **VPC Resolver**
VPCs
Inbound endpoints
Outbound endpoints
Rules
Query logging
Outposts

▼ **Domains**

Records (3)

[Delete record](#) [Import zone file](#) [Create record](#)

Automatic mode is the current search behavior optimized for best filter results. [To change modes go to settings.](#)

[Type](#) [Routing p...](#) [Alias](#)

	Record ...	Type	Routin...	Differ...	Al
☐	sample.com	NS	Simple	-	No

0 records selected

Select a record to see its details

Step 11: Test Auto Scaling

- Generate CPU load on EC2 instances.
- Verify new instances are launched automatically.
- Paste URL of Load balancer Domain name into new web browser to check the application running.
- Check Target Group for Healthy instance running.

The screenshot shows the AWS Management Console for the 'ap-south-1' region. The 'Instances' page is active, displaying a list of EC2 instances. The left sidebar shows the navigation menu with 'Instances' selected. The main content area shows a table of instances with columns: Name, Instance ID, Instance state, Instance type, Status check, and Availability Zone. Two instances are listed: 'myserver8' (Stopped) and 'i-00d47cb4a9a1911ec' (Running). The 'Running' instance has a status check of '3/3 checks passed'.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone
myserver8	i-0ec2a5f45acca9a87	Stopped	t3.micro	-	ap-south-1a
	i-00d47cb4a9a1911ec	Running	t3.micro	3/3 checks passed	ap-south-1a

The screenshot shows the AWS Management Console for the 'ap-south-1' region. The 'Target groups' page is active, displaying details for a target group named 'web-app-tg'. The left sidebar shows the navigation menu with 'Target groups' selected. The main content area shows a 'Details' section with the following information:

- ARN: [arn:aws:elasticloadbalancing:ap-south-1:973629900070:targetgroup/web-app-tg/2fa5c26942aea733](#)
- Target type: Instance
- Protocol : Port: HTTP: 80
- Protocol version: HTTP1
- VPC: [vpc-094e787c6727d5b85](#)
- IP address type: IPv4
- Load balancer: [webloadbalancer](#)

The 'Targets' section shows 1 total target, which is 'Healthy'. The 'Distribution of targets by Availability Zone (AZ)' section shows 0 Anomalous targets.

Target type	Protocol : Port	Protocol version	VPC	IP address type	Load balancer
Instance	HTTP: 80	HTTP1	vpc-094e787c6727d5b85	IPv4	webloadbalancer

Total targets	Healthy	Unhealthy	Unused	Initial	Draining
1	1	0	0	0	0

Application Status: Online

Your highly available, auto-scaling web application infrastructure is successfully serving requests.

Database Endpoint Address : `//amazonaws.com`
Region Zone `ap-south-1` (Mumbai)
Environment Type `AWS Console`
Powered by `EC2 • ALB • Auto Scaling • RDS`

Troubleshooting:

Issue 1: EC2 Instances Not Registering with Load Balancer

Solution:

Verify Security Group rules.

Check Target Group health checks.

Ensure Apache service is running.

Issue 2: Auto Scaling Not Launching New Instances

Solution:

- Verify scaling policies.
- Check CloudWatch metrics and alarms.
- Confirm Launch Template configuration.

Issue 3: Database Connection Failed

Solution:

- Verify RDS Security Group allows MySQL traffic.
- Check database endpoint and credentials.
- Ensure EC2 instances can access private subnet.

Conclusion

The Scalable Web Application with Load Balancing project provides a reliable and highly available cloud infrastructure using AWS services. By integrating ELB, Auto Scaling, EC2, RDS, S3, and Route 53, the system can efficiently manage user traffic, scale automatically based on demand, and ensure secure data management.

The architecture follows cloud best practices including high availability, scalability, fault tolerance, and automation. This solution is suitable for production workloads and can be extended further with CI/CD pipelines, Docker containers, CloudFront CDN, and infrastructure automation tools like Terraform.